



Getting Started

 WIDGET API VERSION 1.4.0

In this article you'll learn what is needed to create an iCUE widget, and how to create your first iCUE widget.

Prerequisites

Developing widgets for iCUE requires:

- **iCUE 5.47** or later installed
- **Text editor**, or preferably a code editor
- **iCUE Widget CLI** for packaging widgets
- **Compatible CORSAIR device** for testing. See [Supported Devices](#) for a list of supported devices.

iCUE Widget CLI

The iCUE Widget CLI is the command-line tool for creating, validating, and packaging widgets before import into iCUE, and is the recommended way to develop for iCUE.

The CLI provides three main commands:

- `icuewidget init` - Creates starter widget files
- `icuewidget validate` - Validates widget structure and manifest
- `icuewidget package` - Creates `.icuewidget` package file



TIP

While you can create widget files manually, the CLI is required for packaging widgets into `.icuewidget` files for distribution and installation in iCUE.

Your First Widget

You can create your first widget in two ways: using the CLI tool or manually creating the files.

Option 1: Using the CLI Tool

If you have the iCUE Widget CLI installed, you can quickly scaffold a new widget:

Terminal

```
icuewidget init MyWidget
```

If you don't have the CLI installed yet, follow the installation guide first:

[Install the iCUE Widget CLI](#)

The CLI will interactively prompt you for:

- Widget name
- Author
- Widget ID (reverse-DNS format, e.g., `com.yourname.mywidget`)
- Description
- Version

This creates a configurable hello-world widget with example properties like headline, subtitle, text color, and background color.

After initialization, you can validate and package your widget:

Terminal

```
icuewidget validate MyWidget  
icuewidget package MyWidget
```

Option 2: Manual Creation

Create a new directory for your widget with the following structure:

```
MyWidget/  
├─ manifest.json  
├─ index.html  
├─ resources/  
│   └─ icon.svg  
└─ translation.json (optional)
```

manifest.json

Create a `manifest.json` file with your widget metadata:

```
{  
  "author": "Your Name",  
  "id": "com.yourname.mywidget",  
  "name": "My Widget",  
  "description": "My first iCUE widget",  
  "version": "1.0.0",  
  "preview_icon": "resources/icon.svg",  
  "min_framework_version": "1.0.0",  
  "os": [  
    {  
      "platform": "windows"  
    }  
  ],  
  "supported_devices": [  
    {  
      "type": "dashboard_lcd"  
    }  
  ]  
}
```



Your widget's ID is a reverse-DNS format string, unique to your widget, that reflects your organization and product, for example:

- `com.corsair.weather`
- `com.yourname.systemmonitor`

Widget IDs must only contain lowercase alphanumeric characters (a-z, 0-9), hyphens (-), and periods (.).

index.html

Create an `index.html` file with your widget's UI and logic:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>tr('My Widget')</title>
    <link rel="icon" type="image/svg+xml" href="resources/icon.svg" />

    <meta
      name="x-icue-property"
      content="textColor"
      data-label="tr('Text Color')"
      data-type="color"
      data-default="'#FFFFFF'"
    />

    <script type="application/json" id="x-icue-groups">
      [{ "title": "tr('Settings')", "properties": ["textColor"] }]
    </script>
  </head>
  <body>
    <div id="content">Hello World</div>
    <script>
      icueEvents = {
        onICUEInitialized: init,
        onDataUpdated: update,
      };

      function getIcueProperty(name) {
        if (typeof window !== "undefined" &&
Object.prototype.hasOwnProperty.call(window, name)) {
          const value = window[name];
```

```

        if (value !== undefined && value !== null && value !== "") return
value;
    }
    try {
        const value = Function('return typeof ' + name + ' !== "undefined" ?
' + name + ' : undefined')();
        if (value !== undefined && value !== null && value !== "") return
value;
    } catch (e) {}
    return undefined;
}

function init() {
    update();
}

function update() {
    const color = getIcueProperty("textColor") || "#FFFFFF";
    document.getElementById("content").style.color = color;
}

if (iCUE_initialized) init();
</script>
</body>
</html>

```

If your widget uses `tr()`, add a `translation.json` file in the widget root. More info in [Translations](#)

Packaging Your Widget

Once you have your widget files ready, you need to package them into an `.icuewidget` file using the CLI:

Terminal

```
icuewidget package MyWidget
```

This command will:

- Validate your widget structure and manifest

- Create an `.icuewidget` package file
- Display a summary of the packaging process

Loading in iCUE

To install your widget in iCUE:

1. Open iCUE
2. Navigate to the widgets section
3. Click the + button above the list of available widgets
4. Select your `.icuewidget` file
5. Your widget should now appear in the list and be ready to use

Supported Devices

iCUE widgets can run on devices with LCD displays:

Device Type	Examples
<code>dashboard_lcd</code>	XENEON EDGE
<code>keyboard_lcd</code>	VANGUARD 96, VANGUARD PRO 96
<code>pump_lcd</code>	iCUE LINK XC7 ELITE LCD, iCUE LINK XD5 ELITE LCD

What's Next?

- Learn about the [Widget Specification](#) for building widgets
- Explore [Controls](#) for creating interactive settings in the iCUE settings panel
- Discover the [iCUE Global Object](#) for utility functions and system information
- Access [Device Information](#) to identify the device displaying your widget
- Use [Plugins](#) to access system sensors, media playback, and more

- Add [Translations](#) to support multiple languages